Fuji Manual

v10.1.2-ff0392ea67-mc1.21.7

sakurawald

July 9, 2025

Contents

Chapter 1

Concept

1.1 Configuration File

1.1.1 Definition

All `files` inside `config/fuji/` directory are named `configuration files`

1.1.2 Types

Note: The types of configuration files

Control File A `control file` is used to control behaviours.

- **Main-Control File** The **main-control file** refers to the **config/fuji/config.json** file, which is used to enable/disable a module.
- **Module-Control File** Some modules have their own control files, which is used to control the behaviour of that module.

User-Data File User-data files are used to store the data generated by the user.

1.2 Module

1.2.1 Definition

A `module` is used to provide a specific purpose.

Example: The purpose of modules

Chat Module provides chat-related customization.

Tpa Module provides `/tpa` command.

1.2.2 Properties

The key properties of a module:

A module can be disabled You can disable a module completely in `main-control file` by setting the value of its `enable` key to `false`.

A module can work independently You can enable or disable a specific module individually.

1.2.3 Module Path

A `module` is identified by a unique `module path`.

Example: What a module-path looks like?

The module path of the module `tpa` is `"tpa"`.
The module path of the module `history` whose parent module is `chat`, is `"chat.history"`.
You will see a list of `enabled modules` identified by their `module path` at the server-startup process.

A `module` can have `sub-module`. The relationship between `parent-module` and `sub-module` is relative, and there is nothing special about `sub-module`.

1.2.4 How to enable/disable a module?

You can enable/disable a module in `config/fuji/config.json` file by setting the value of its `enable` key to `true`/`false`, and re-start your server to apply the changes.

A `module` will be enabled if the following conditions are met:

1. The value of `core.debug.disable_all_modules` is `false`.
2. The required dependency mods are installed.
3. Its `parent-module` is `enabled`.
4. The value of its `enable` key is `true`.

Example: How to enable a sub-module?

To make the module `chat.display` enabled, you need to enable `chat` module first.

Tip: How to know what modules are enabled?

Issue `/fuji` command, to see what modules you have enabled.

1.3 Job

1.3.1 Definition

A `job` is some things that will be done `repeatedly`.

1.3.2 Cron Expression as Trigger Rule

A language named `cron language` is used to `define` when a `job` should be triggered.

Tip: Don't write cron expression by hand. Use generator!

An example `cron expression` `"0 * * ? * *"`, means `"trigger the job every minute"`.
You can use the generator to generate a `cron expression`: https://www.freeformatter.com/cron-expression-generator-quartz.html

1.3.3 List all scheduled jobs

Tip: How to query the scheduled jobs?

Issue `/fuji inspect jobs` command, to see all the scheduled jobs.

1.4 Regex

1.4.1 Definition

Regex is a language used to define the pattern of strings.

The typical use-case is to define the **pattern string**, and use it to match the given **input string**.

1.4.2 Reference

You can write **pattern string**, to match a given **input string** instance. Use the following online editor, to test your **pattern string**.

1. https://regexr.com/
2. https://regex101.com/

1.5 Placeholder

1.5.1 Definition

A `placeholder` is a string which will be replaced based on context.

Tip: What is the placeholder api in fabric platform?

There is a plugin named `PlaceholderAPI` in bukkit platform.
Also, there is a mod named `Text Placeholder API` in fabric platform.
They are different projects, but provides the same purpose.

1.5.2 Example

Example: Replace player name by context

The placeholder `%player:name%` will be replaced by the name of the contextual player.

1.5.3 Reference

1. https://placeholders.pb4.eu/user/default-placeholders/
2. https://placeholders.pb4.eu/user/mod-placeholders/

1.6 Identifier

1.6.1 Definition

An `identifier` is a `string` to name an `object` in `Minecraft`.

1.6.2 Example

Example: The identifier of items

The identifier of apple is `"minecraft:apple"`.
The identifier of diamond is `"minecraft:diamond"`.

Example: The identifier of entities

The identifier of zombie is `"minecraft:zombie"`.

Tip: Query a registry for all identifiers.

Issue `/fuji inspect registry` command, to see all the registries in Minecraft.

1.6.3 How to query all identifiers of a specific type

Example: Query all identifiers of entity

```
/summon ...
```

Example: Query all identifiers of block

```
/setblock ...
```

Example: Query all identifiers of item

```
/give ...
```

Chapter 2

Command

2.1 Definition

A `command` is used in Minecraft to perform a specific action/function.
Fuji provides lots of commands for various purpose.
Commands are registered by a module.
By default, all modules are disabled, so you have to enable modules in `config/fuji/config.json` file, and re-start your server to apply the changes.

2.2 What are the commands provided by this mod?

Issue `/fuji` command, to list all the enabled modules.
Issue `/fuji inspect fuji-commands`, to list all the registered commands by enabled modules.

Chapter 3

Permission

3.1 Definition

A `permission` is used to decide whether `a player` can do something or not.

3.2 Types

To make the discussion clearer, we define the types of `permission` as follows:

level permission A permission level is a non-negative number used in vanilla Minecraft. The higher number means the higher authority.

string permission Usually, a `string permission` is introduced by a `permission plugin`, such as `luckperms` mod.

3.3 What is the permission system used by fuji?

3.3.1 Explanation

Fuji use the mojang's `vanilla Minecraft permission system`, which is based on `level permission`.

As a convention, most of the commands registered by fuji, requires `level permission` to be `0` to use. Only a few of the commands require the `level permission` to be `4` to use.

Tip: Modify the default required level permission to be 4 for all fuji commands

You may want to prohibit the player from using any commands registered by fuji. And then assign the proper permission for each command one by one. In this case, you can set the `"all_commands_require_level_4_permission_to_use_by_default"` in `config/fuji/config.json` file to be **true**.

3.3.2 Set a string permission for a command

By default, fuji only use the `level permission` as `the requirement of` a command. However, if you want to use `string permission` for a `command`, you can use **command_permission** module, to provide luckperms permissions for each command.

Tip: Query all registered string permissions.

Issue `/fuji inspect permissions-and-metas` command, to see all the registered permissions.

3.4 Reference

1. https://minecraft.fandom.com/wiki/Permission_level
2. https://luckperms.net/wiki/Home

Chapter 4

Meta

4.1 Definition

A `meta` is a `key-value pair`.

Note:

Note that `meta` is introduced by `luckperms` mod, there is no `meta` in vanilla minecraft.

4.2 Getting Started

Tip: List all supported metas.

Issue `/fuji inspect permissions-and-metas` command.

4.3 Example

Example: Set a meta for a gruop.

The `home module` supports the meta `fuji.home.home_limit`, which controls how many homes a player can create. To set the max homes limits to `3`: `/lp group default meta set fuji.home.home_limit 3`

Example: Query all metas for a group.

```
/lp group default info
```

Tip: Query all registered metas.

Issue `/fuji inspect permissions-and-metas` command, to see all the registered metas.

Chapter 5

Configuration

5.1 How to see the document for each field in config file?

Issue `/fuji inspect configurations` to inspect configuration files. Click the interested configuration file, to see detailed document for it.

Chapter 6

Language

6.1 Definition

The files inside **config/fuji/lang/** directory are called **language files**.
Each **language file** defines a table to map a **language key** into a **language value**.

6.2 Modify the default language used by fuji.

You can modify the **default language** in `config/fuji/config.json` file. After that, issue `/fuji reload` command to apply the changes.

6.3 Control how the text is sent.

The **language value** only defines what is the text. It didn't define how to use that text. You can use **language instructions** to specify how to use that **language value**.
Currently supported language instructions:

1. **Send the text via action bar** If the language value starts with `"[send-action-bar]"`
2. **Send the text via main title** If the language value starts with `"[send-main-title]"`
3. **Send the text via sub title** If the language value starts with `"[send-sub-title]"`
4. **Suppress the sending of this text** If the language value starts with `"[suppress-sending]"`

If no **language instruction** is used, we will send the **text** via the **message**.

6.4 Enable multiple language support and respect the client side language.

You can enable the `"language"` module, to provide the multiple language support for each player, and respect their client side language option.

Chapter 7

Q&A

7.1 Where is the configuration files of fuji?

As a convention, all the files are placed in `config/fuji/` directory.

7.2 What is .json file?

A json file is a text file, whose file name normally ends with **.json**.

7.3 How can I edit a configuration file?

To ensure the `readibility` and `compatibility`, most of the files are saved as `pure text format`. You can open them with a `text editor`. Remember to issue `/fuji reload` command to apply your modification.

Tip: Use a moderen text editor.

Some files may have a large number of lines, so it's highly recommended to use a `modern` text editor, which can highlight symbols and reveal the structure of the file, such as:

1. Visual Studio Code
2. Visual Studio Code - Web Online Editor
3. Vim
4. Emacs
5. Sublime Text

7.4 What is .dat file?

The file whose name ends with **.dat** are the `vanilla minecraft NBT format file`. To open such a file, you need to use a `NBT Editor`, such as `NBTExplorer`.

7.5 How to update fuji to a new version?

Please follow the steps:

1. **Backup the data** Back up the `config/fuji` directory.
2. **Test the new version in your test-environment** Put the new version of fuji into `mods/` directory, start the server, and adjust the configuration to your desired effect.
3. **Apply the changes to production-environment** Now, it's ready to apply the changes to your production-environment.

Warning: Don't test new changes directly in your production-environment

It's highly recommended to setup a test-environment for a network-maintainer, so that you can test and tweak installed mods into the desired effect, and avoid un-expected situations.

7.6 Fuji conflits with one of my mods.

You can `disable` that `conflicting module` and re-start your server. If possible, create an issue at fuji issue page, so that we can solve it later.

7.7 Can I ask the forge or neoforge support?

We have no plan for forge or neoforge platform. However, you can try running this mod via **sinytra-connector** mod. It's tested in the following environment (105/105 modules works):

```
Loader: Forge mod loading, version 47.4.0, for MC 1.20.1 with MCP
↪  20230612.114412
Mods:
- connector-1.0.0-beta.46+1.20.1
- forgified-fabric-api-0.92.2+1.11.12+1.20.1
- fuji-fabric-7.4.0-b93ba03936-mc1.20.1
```

If you encounter issues in forge or neoforge environment, you can still open an issue in github. We provide the support for forge and neoforge, if it's possible and handy.

7.8 How can I report bugs or suggest new features?

You can create an issue at fuji issue page

Chapter 8

Suggestion

8.1 Explanation

Here is a collection of server-side mods that existing and recommended to use, which can make your life easier in fabric server-side admin.

Note that fuji doesn't require these mods installed to work, and some of these mods have the same functionality as fuji.

8.2 Useful server-side mods

1. maintenance mode
2. backpacks polymer
3. seed guard
4. spawn anywhere
5. world threader
6. showcase fabric
7. filament
8. poly nutrition
9. camera boscura
10. toms mobs
11. sand storm
12. world manager
13. world game rules
14. holo displays
15. offline commands
16. custom name tags
17. easy whitelist
18. custom name
19. trinkets
20. direction hud
21. pet protect
22. essential addons
23. easy auth
24. fast back
25. patbox's brewery
26. mine spawners
27. yet another world protector
28. litematica server paster

29. secure crops
30. fsit
31. mob filter
32. blue map
33. anti xray
34. melius commands
35. vanilla permission
36. vanish
37. villager config
38. world-edit
39. essential commands
40. missions
41. luckperms
42. tab
43. armor stand editor
44. ban hammer
45. image2map
46. polydecorations
47. sleep warp
48. styled chat
49. styled nickname
50. styled player list
51. styled sidebar
52. universal graves
53. universal shop
54. goml
55. polydex
56. polymania
57. head index

58. inv view
59. ledger
60. falling tree
61. double doors
62. skin restorer
63. husk homes
64. yet another world protector
65. krypton
66. sit
67. stack deobf
68. lithium
69. let me despawn
70. carpet
71. mod viewer
72. simple voice chat
73. mini motd
74. tab tps
75. spark
76. polyfactory
77. ouch
78. chunky
79. afk plus
80. taterzens